This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

Reinforcement Learning Assisted Multi-UAV Task Allocation and Path Planning for IIoT

Guodong Zhao, *Student Member, IEEE,* Ye Wang, Tong Mu, Zhijun Meng, Zichen Wang

***Abstract*—Exploring the widespread applications of unmanned aerial vehicles (UAVs) in Internet of Things has become a current research hotspot. In some tasks related to UAV-based environmental monitoring and transportation, the simultaneous consideration of UAV task allocation and path planning constitutes a category of joint optimization problems. This paper focuses on a warehouse cargo inspection scenario with multiple heterogeneous UAVs. In such scenarios, existing heuristic path finding algorithms that consider task allocation cannot make a good balance between solution time and solution quality. Therefore, in this paper, we propose a reinforcement learning assisted task allocation and conflict-free path framework to achieve better task allocation and path finding results. The framework uses a multiple traveling salesman transformation algorithm for task allocation and a multi-agent reinforcement learning (MARL) algorithm for conflict-free path finding. The path finding policy can be extended to larger-scale environments with more UAVs. We conduct the training of the path planning module and the verification of the overall framework in random environments. Simulation results show that our reinforcement learning assisted framework has a significant advantage over the existing algorithms in terms of solution time, solution quality and scalability.**

***Index Terms*—Multi-agent reinforcement learning (MARL), task allocation, multi-agent path finding, unmanned aerial vehicle (UAV).**

I. Introduction

THE application of unmanned aerial vehicle (UAV) in industrial Internet-of-Things (IIoT) is becoming increasingly widespread due to its flexible deployment, high mobility and diverse functionalities, such as in the field of intelligent transportation [1], intelligent detection [2] or auxiliary communication [3]. More and more complex tasks in these field require heterogeneous UAV clusters to complete with collaboration. IIoT such as UAV-based scenarios have multi-functionality, but system design needs to consider the unity of logical abstraction [4]. The UAV traffic management [5], [6] is used as an example, which consists of a central service system and individual modules installed on each UAV. The central service module classifies the functions and status of UAVs, responsible for unified management of UAVs, including task allocation, data aggregation, etc. The standalone module of the UAV is responsible for receiving tasks issued by the central service module, exchanging data with the central service module, and executing specific tasks based on its own functional modules. In addition, in some communication-limited scenarios [7]–[9], UAVs can only rely on their own sensors for perception and automated flight. These sensors may include LiDAR, depth cameras, video, ultrasound.

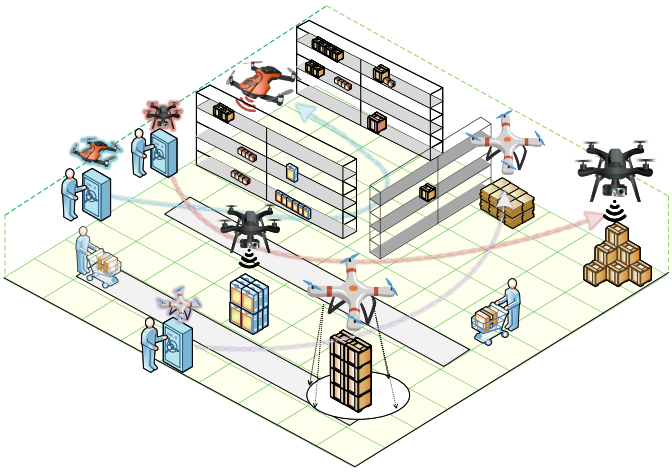


Fig. 1: The illustration of heterogeneous UAVs in a warehouse monitoring task scenario, where heterogeneous UAVs are equipped with different sensors corresponding to different tasks. The UAV warehouse management system performs task allocation and path finding based on the functions of UAVs and existing tasks in the warehouse.

In this paper, we consider a UAV-based environmental detection scene in a hazardous goods warehouse, as shown in Fig. 1, multiple UAVs equipped with various sensors need to jointly measure temperature, humidity, and other indicators, and detect potential leaks of various hazardous chemicals at many task target locations. This requires the management system to plan the path of the UAVs, so that each target location is accessed by the UAV at least once. Furthermore, their paths cannot conflict. As UAVs are heterogeneous and

This work of Zhijun Meng was partly supported by the National Natural Science Foundation of China under Grant No.61976014, and partly supported by the Aeronautical Science Foundation of China under Grant No.2022Z071051001. (Corresponding author: Zhijun Meng.)

G. Zhao, Z. Meng and Z. Wang are with the School of Aeronautic Science and Engineering, Beihang University, Beijing 100191, China (e-mail: {zhaoguodong00, mengzhijun, wangzichen}@buaa.edu.cn).

Y. Wang is with the School of Computer and Artificial Intelligence, Zhengzhou University, Zhengzhou, 450001, China (e-mail: iswangye@gs.zzu.edu.cn).

T. Mu is with the Department of Automation, North China University of Technology, Beijing, 100144, China (email: phdmutong@mail.ncut.edu.cn).

Copyright (c) 2024 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

equipped with various sensors, only a subset of UAVs have corresponding sensors to detect the expected data at the target location, this introduces task allocation constraints that must be followed before planning paths.

We model the joint problem of task allocation and path planning for multiple heterogeneous UAVs as a multi-agent combinatorial path finding (MCPF) problem [10]–[12], as shown in Fig. 2. The system must compute conflict-free paths for all agents, while ensuring that they visit intermediate task target locations before reaching their destinations and meet task assignment constraints. The existing systems and algorithms [10], [13] for the MCPF model use a centralized method that couples task allocation and path finding in a central module for all UAVs. The existing work cannot make a good balance between solution quality and solution efficiency. To get better solution quality with higher probability, it often needs to carry out multiple rounds of iterative search, which is very inefficient; whereas, to obtain a better solution efficiency, the solution quality cannot be guaranteed. In addition, the task assignment and path finding module are coupled seriously, so to facilitate the path finding process, the optimal task assignment result is often compromised.

Learning-based methods are gradually demonstrating significant advantages in the field of path finding problems. In particular deep reinforcement learning (DRL) method has shown good performance in various planning tasks such as autonomous driving [14], robotics [15] and UAVs [16], as they are very effective in the inference process with constant level time complexity. Compared with traditional search-based or heuristic-based path finding methods, DRL has better generalization and scalability in some complex environments due to the high representation ability of neural networks [17]–[19]. In order to handle path conflicts between multi-agent systems, multi-agent reinforcement learning (MARL) algorithms can be designed to do path finding.

To solve the joint optimization problem of task allocation and path finding, we propose a reinforcement learning assisted path finding framework. The framework consists of a centralized task allocation module and a multi-agent path finding (MAPF) module. In the task allocation module, a transformation method based on distance-weighted multiple traveling salesman problem (mTSP) is used to assign tasks, which can ensure that the UAVs obtain the shortest path by following the task sequence. The conflict-free path trajectory between multiple UAVs task locations is controlled by the policy obtained by the multi-agent proximal policy optimization (MAPPO) algorithm. By using the MARL algorithm to obtain conflict-free path trajectories between multiple UAVs task locations, it has a good time efficiency in path finding process. Besides, the path finding strategy trained in a small number of UAV environments can be extended to any number of UAV environments.

Overall, the contributions of this paper are as follows:

1) An efficient multi-agent task allocation and path finding framework assisted with reinforcement learning is proposed. The mTSP transformation method with distance as the weight can ensure the optimality of task allocation considering the path length. The path search policy based on MARL can enable UAVs to obtain the optimal conflict-free path as quickly as possible.
2) In the MARL-based path finding module, in order to ensure the safety of the decentralized path finding policy, the temporal environment information around each UAV is encoded to handle static and dynamic obstacles during planning. The RL assisted path finding framework can be extended to any environment with different numbers of UAVs and tasks.
3) Simulation experiments show that compared to existing algorithms, the reinforcement learning assisted framework proposed has great advantage in solution time efficiency while ensuring better solution quality, especially in large-scale problem scenarios.

The subsequent sections of this paper is shown as follows. Section II introduces related work on joint path finding based on heuristics and learning methods. In Section III, we give the problem formulation of task assignment and path finding based on the UAV warehouse detection scenario. Section IV gives the preliminary definition and algorithm, the detailed design of MARL assisted algorithm is given in Section V. Finally, the simulation experiment and conclusions of the proposed algorithm are shown in Section VI and Section VII, respectively.

II. RELATED WORK

In recent years, most studies have proposed various heuristic algorithms and swarm intelligence optimization algorithms [20]–[24] to provide solutions for task allocation and path finding problems. Moreover, some system frameworks [25], [26] based on easily computable objective functions have been proposed to solve the multi-UAV collaborative joint task allocation and path finding problems. Among the swarm intelligence algorithms, the ACO algorithm is a widely used optimization method for path finding, which can plan the optimal collision-free path, obstacle avoidance, and coverage tasks in complex environments [27]–[29]. However, all these works focus on static environments and ignore the dynamic obstacle scenarios. Furthermore, the multi-agent pickup and delivery problem [30]–[32] computes a set of collision-free paths to complete a set of extraction and delivery tasks, and also requires assigning a series of tasks to each agent. Existing methods either brute-force enumerate all possible task sequences in the path finding process to ensure the optimality of the solution [33], [34], posing a computational burden for scalability, or fail to ensure the solution quality.

However, some people suggest that path finding problems can be solved in a totally end-to-end manner [33], [35], [36] based on learning methods. For MAPF, PRIMAL [19] uses image-based representations and targets as input sources. However, they did not consider non-cooperative dynamic obstacles and time information in their work. Liu et al. proposed a decentralized partially observable MAPF method with evolutionary reinforcement learning [17], which aims

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

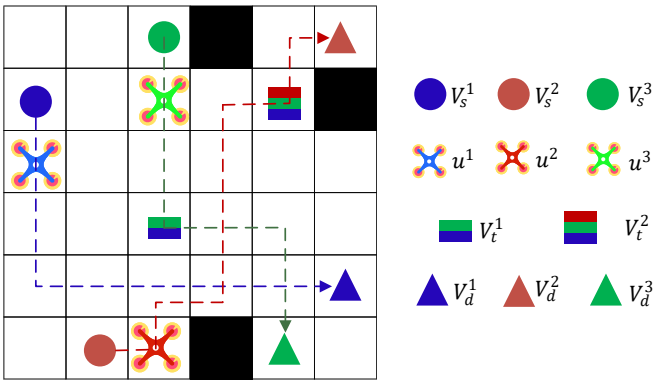


Fig. 2: A feasible trajectories for heterogeneous UAVs based on MCPF model. Three heterogeneous UAVs $\{u_1, u_2, u_3\}$ are considered and their starting locations are $\{v_s^1, v_s^2, v_s^3\}$, and destinations are $\{v_d^1, v_d^2, v_d^3\}$, while task targets are $\{v_t^1, v_t^2\}$. The color of the targets and destinations indicates the assignment constraints, i.e., the subset of UAVs that are eligible to visit the target or destination. For example, the task in target v_t^1 can be done by either the green or blue UAV. Feasible trajectories for UAVs are shown using dashed lines.

to learn effective local planning strategies in mixed dynamic environments. This method models dynamic obstacle behavior using image-based representations and decomposes long-range navigation tasks into many easier waypoint condition subtasks. In the work of [37], a Q-learning based approach is used for obstacle avoidance of single UAV, without modeling multi-UAV scenarios.

III. PROBLEM FORMULATION

In this paper, we consider a scenario where heterogeneous UAVs are used to detect hazardous material leakage, cargo temperature and humidity with other cargo information in a warehouse. And each UAV is equipped with different sensors to perform multiple detection tasks. Heterogeneous UAVs need to complete all tasks in the shortest possible time, and their paths should be collision free. The layout of the warehouse and the hazardous materials forms a map as shown in Fig. 2, in which there are n UAVs $\{u_i, u_2, u_3, ..., u_n\}$ that can perform tasks together, the starting locations for each of them are $V_s = \{v_{s1}, v_{s2}, v_{s3}, ..., v_{sm}\}$, also the corresponding destinations are $V_d = \{v_{d1}, v_{d2}, v_{d3}, ..., v_{dn}\}$. In the warehouse, there are m hazardous material target locations $V_t = \{v_{t1}, v_{t2}, v_{t3}, ..., v_{tm}\}$ that need to be detected by UAVs. For each target location v_{di}, only the specified UAV can complete the detection task, so we define the function $c_i = f_A(v_{ti})$, where c_i represents the subset of UAVs that can detect the target location v_{ti}. Finally, we use an finite undirected graph $G = (V, E)$ to represent the total workspace where $V = V_s \cup V_d \cup V_t$ and $E = V \times V$.

Note that the focus of this paper is to investigate the feasibility of using a new framework to solve the joint problem of path finding and task allocation, considering the 3D environment of UAV flight height will only increase the complexity of the UAV kinematics and dynamic model, therefore, in order to simplify but not lose generality, we assume the height of all UAVs is constant, so the entire map can be represented by a 2D plane with a dimension of $x \times y$. In addition, the speed and direction of the UAV at time t are defined as $[v_x, v_y, cos\theta, sin\theta]_t$, where$[v_x, v_y]$is the partial velocity of the UAV along the x-axis and y-axis directions, and $[cos\theta, sin\theta]$ is the corresponding heading angle; the time from takeoff to landing of UAVs is $\{T_1, T_2, T_3, ..., T_n\}$, where T_i is the total flight time of the i-th UAV.

We assume that UAVs are equipped with detectors that can detect both static and dynamic obstacles (e.g. cargo shelves, transport vehicles or other UAVs), and the obstacle is defined as $\mathcal{I} = \{\{\text{static obstcals}\} \cup \{\text{mobile obstcals}\}\}$. For safety reasons, it is required that UAVs should not collide with static obstacles and other flying UAVs. We define the safe distance as ϵ, which is determined by the environment available for safe flight of the UAV and the distance sensors provided by itself. When the Euclidean distance between a UAV and other objects is less than ϵ, a collision occurs, which is defined as an unsafe state. Assuming $p_u^i(t) = (x_u^i(t), y_u^i(t))$ and $p_o^i(t) = (x_o^i(t), y_o^i(t))$ represent location of UAV itself and the location of obstacles it detects at time t, therefore the collision-free safety state can be defined as

$$s(p_u^i(t)) = \begin{cases} 1, & \text{if } \forall \quad o \in \mathcal{I} \quad \text{where} \quad \delta_u^i > \epsilon, \\ 0, & \text{otherwise,} \end{cases} \tag{1}$$

where

$$\delta_u^i = \sqrt{\left(x_u^i(t) - x_o^i(t)\right)^2 + \left(y_u^i(t) - y_o^i(t)\right)^2}. \tag{2}$$

Assuming $T(\cdot)$ is the maximum time at which all UAVs reach the destination location, and all UAVs' trajectories are defined as

$$\tau = \{\tau_i | \tau_i = (p_u^i(0), ..., p_u^i(T_i))\}. \tag{3}$$

Therefore, $T(\tau)$ corresponding trajectories τ can be defined as

$$T(\tau) = \max(t_i), i = 1, 2, ..., n. \tag{4}$$

When the UAV reaches the destination location, it is necessary that all predefined task target locations have been accessed by the desired UAVs, so every target location should be included in the UAVs' trajectories, therefore task constraints can be defined as

$$P(v_{ti}) = \{\tau_j | \forall u_j \in c_i\}, \tag{5}$$

where $P(v_{ti})$ represents the two-dimensional coordinates of target position v_{ti}.

Sensors carried by UAVs require a certain amount of time to complete the detection task, so we define that at each task location v_{ti}, the UAV to perform task in c_i need to spend time $g(v_{ti})$ to finish the task. We assume that the UAV is in a hovering state during task execution, and at each task location, the hovering time $h(P(v_{ti}))$ of the UAV is:

$$h(P(v_{ti})) \geq g(v_{ti}). \tag{6}$$

In summary, the task allocation and path finding problem on heterogeneous UAV environment can be modeled as:

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

$$
\begin{aligned}
&\underset{\tau}{\text{argmin}}\ T(\tau) \\
\text{s.t.}\quad & p_u^i(t=0) = v_{si} \quad i=1,2,...,n, \\
& p_u^i(t=T_i) = v_{di} \quad i=1,2,...,n, \\
& s(p_u^i(t)) = 1 \quad \forall t < T_i, \quad i=1,2,...,n, \\
& h(P(v_{ti})) \geq g(v_{ti}) \quad i=1,2,...,n, \\
& P(v_{ti}) = \{\tau_j | \forall u_j \in c_i\} \quad i=1,2,...,n,
\end{aligned} \tag{7}
$$

in which the objective function aims to obtain the optimal path of all UAVs to minimize the maximum time of $T(\tau)$. The first and second constraints are the initial and final positional constraints, which constrain the UAV to be at the starting location at the initial time and at the destination locations at the end time of T_i.

IV. Preliminaries

A. Marko Decision Process

The path finding process of multiple heterogeneous UAVs can be formulated as a partially observable Markov decision process (POMDP), which is defined by the tuple $< A, S, O, R, T, \gamma, \rho_0 >$. Here A stands for the action space, O represents the observation space and S denotes the state space; R:$S \times A \to \mathbb{R}$ is the reward model, T:$S \times A \times S \to \mathbb{R}$ is the transition model; $\gamma \in [0, 1)$ is the discount factor, ρ_0 is the initial state distribution. At each step t the agent is in a state s_t, and every agent i chooses an action a_t^i based on its policy $\pi_i(a^i|s_t)$ and receives the reward $R_i(s_t^i, a_t^i)$. Subsequently, the environment transits to a new state $s_{t+1}^i \sim p(|s_t^i, a_t^i)$. The objective function of a POMDP model is to find a optimal policy π_i that maximizes the return as

$$
\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{s_0, a_0, o_0, ...} \sum_{t=0}^{\infty} \gamma^t R_i\left(s_t^i, a_t^i\right). \tag{8}
$$

B. Trust Region Policy Optimization

The objective of POMDP is to find the optimal policy. Policy optimization are one of such method to achieve this goal.In policy optimization methods, the policy is represented as a random parameterized function or neural network $\pi(\theta)$, so the objective function of the POMDP becomes

$$
\theta^* = \arg\max_{\theta} J(\pi(\theta)), \tag{9}
$$

where $J(\pi(\theta)) \triangleq \mathbb{E}_{\mathbf{a_t} \sim \pi(\theta)} [\sum_{t=0}^{\infty} \gamma^t R(\ \mathrm{s}_t, \mathbf{a}_t)]$, which is aimed to maximise the total reward. And the policy are usually optimized by gradient ascent iterative approach

$$
\theta_{k+1} = \theta_k + \alpha \left.\nabla_\theta J(\pi_\theta)\right|_{\theta_k}, \tag{10}
$$

which keeps new and old policies close(every iterative step $\alpha \left.\nabla_\theta J(\pi_\theta)\right|_{\theta_k}$) in parameter space. But even seemingly small differences in parameter space can have very large differences in performance, which makes it dangerous to use large step sizes thus hurting sample efficiency. TRPO are introduced to nicely avoid this kind of collapse, which is defined as

$$
\begin{aligned}
&\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi) \\
&\text{s.t.} \quad D_{KL}\left(\pi, \pi_k\right) \leq \delta,
\end{aligned} \tag{11}
$$

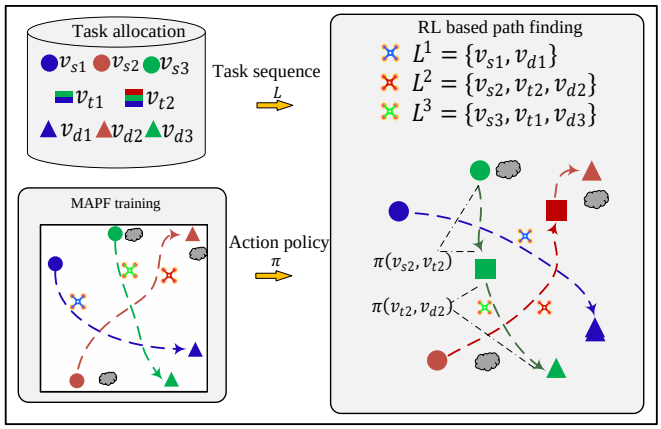


Fig. 3: System framework. It mainly consists of three modules: an offline RL-based MAPF training process, which trains the path-finding policy π, an online task allocation and path-finding process, where each UAV is assigned a task sequence L_i. During the path-finding process, each UAV sequentially accesses its corresponding task target locations in the task sequence controlled by the policy network π.

where $D_{KL}\left(\pi, \pi_k\right)$ is average Kullback-Leibler divergence to measure how close the new and old policies are allowed to be. By this constraint, TRPO can update policies by taking the largest step possible while still can improve performance.

V. MARL Assisted Algorithm Design

The system model proposed in this paper is structured into three distinct modules: task allocation module, MAPF training module, and MAPF execution module. Initially, the task allocation module assigns tasks to each UAV utilizing the mTSP transformation method. Subsequently, each UAV navigates the task points sequence, guided by the action policy acquired during MAPF training. The training phase, conducted offline, entails uninhibited exploration to develop a policy that ensures conflict-free path finding. Consequently, the online execution phase encompasses both task allocation and reinforcement learning-based path finding, which adheres to the outcomes of task allocation. The structure of the entire system is shown in Fig. 3.

The task allocation stage takes the starting locations v_s, destination locations v_d, target locations v_t of all UAVs and corresponding task allocation constraints c_i as inputs. Then it will produce task allocation results as an joint task sequences. Each UAV has a corresponding task sequence $L^i = \{v_{si}, v_{ti_1}, v_{ti_2}, ..., v_{di}\}$, which needs UAV to follow sequentially throughout the entire process. The first item in each sequence is the starting locations v_{si} while the last item is the destination v_{di}, besides, the middle trajectory points $v_{ti_1}, v_{ti_2}, ...$ are the task target locations.

After obtaining the target task sequence of all UAVs from the task allocation module, each UAV only needs to follow the order of the task sequence. Between every two task locations in the middle, the UAV will use reinforcement learning-based strategies for specific flights, ensuring the fastest possible

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

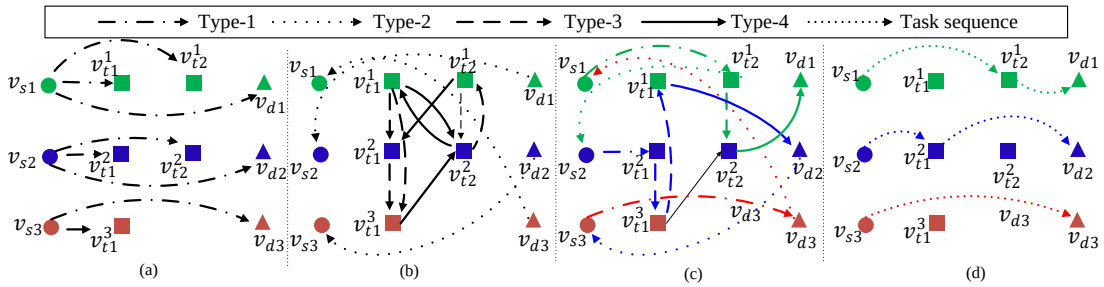


Fig. 4: Transformation method for the example in Fig. 2. (a) and (b) are edges constructed by the transformation rule, where some edges from the start location to destination are omitted to make the plot readable. (c) a single agent TSP solution. (c) multi agent task allocation sequences

flight speed and avoiding collisions with other static or dynamic obstacles. Once the UAV reaches a target allocation and finishes the task, it needs to immediately switch its starting allocation to the current target allocation and its next target task allocation in the task sequence to the destination allocation. During the path finding period, all UAVs do not need to synchronize to make common decisions, as it is trained by an asynchronous MARL learning framework.

A. Target Constrained Task Allocation

The task allocation model in this paper is a transformation method [38] used for the mTSP in [10] with task allocation constraints. The main idea of the transformation method is to convert mTSP problem into a single agent TSP problem and solve it using existing efficient TSP solving algorithms. Then the solution of the single agent TSP will be converted back into the original solution of mTSP, which is also the optimal solution of mTSP. In origin mTSP problems, each target task location can be accessed by any agent as long as there is a path between them, but in the model with task allocation constraints, each target task location must be accessed by at least one UAV in the task constrained set c.

In the process of the ransformation method, first it constructs a transformed graph $\hat{G} = \{\hat{V}, \hat{E}, \hat{C}\}$ based on graph G and target allocation constrain f_A. Let v^i denote a copy of $v \in V_t$ for agent i, if $i \in f_A(v_{ti})$. The vertex set is defined as $\hat{V} := V_s \cup \{U_i | i = 1, 2, \ldots, n\}$ and U_i represents the set of all copies of targets and destinations that agent i is eligible to visit according to f_A. Besides, let $U(v)$, $v \in V_t \cup V_d$ denote an ordered sequence of all copies v^i of vertex v, where the order is specified by sorting i from the smallest to the largest. Based on this order in $U(v)$, let $Next(v^i)$, $v_i \in U(v)$ denote the next copy of v in $U(v)$ and let $Pre(v^i) \in U(v)$ denote the previous copy of v in $U(v)$. The type of edges $(\hat{v}_i, \hat{v}_j)$ and corresponding cost $\hat{c}(\hat{v}_i, \hat{v}_j)$ are defined as

1) Type-1: $\hat{C}(v_{si}, u_i) = C(v_{si}, u^i), \forall u_i \in U^i$,
2) Type-2: $\hat{C}(v_{si}, v_d^{i+1}) = 0$,
   $\hat{C}(v_{sn}, v_d^1) = 0$,
3) Type-3: $\hat{C}(v^i, Next(v^i)) = 0$,
4) Type-4: $\hat{C}(Prev(v^i), u_i) = Z + C(v^i, u^i), \forall u_i \in U^i$,

where $C(u, v)$ represents the shortest path length between vertex (u, v) and $Z = 2(N + M)max_{(u,v)\in E}C(u, v)$. The example of Type-1 edges is shown in Fig. 4(a), Type-3 and Type-4 edges are shown in Fig. 4(b).

After creating the graph G_T, a standard TSP model can be established and then solved using the TSP solver such as LKH. The example result of this TSP problem is shown in Fig. 4(c). Then the single-agent TSP solution can be divided into N segments by removing Type-2 edges in the solution, and each segment tour is a task sequence list π_i which belongs to agent i.

The property of this transformation method between mTSP and TSP can be fully summarized in theorem 1. The detailed proof can be found in [10], [39].

Theorem 1. *Given G and task allocation constrains f_A, the method based on transformation of mTSP finds a minimum cost sequence can arrive at the destination, while visiting all the targets locations and satisfying all the task constraints.*

B. MAPPO based MAPF

As mentioned in system model, each UAV only needs to control its flight in every subinterval of sequences, so the entire process can be seen as the MAPF problem, where the reinforcement learning-based method only needs to control all UAVs start from their start and reach destination as quickly as possible but with no collision. The proposed multi-UAV path finding module based on MAPPO algorithm is shown in Fig. 5. MAPPO is the main framework, while state space prepossessing and reward design are also important.

1) MAPPO based Problem Statement: As mentioned in the definition of POMDP, the detailed design of the MAPPO based path finding system is divided state space design, action space design and reward design:

State space: In the path finding environment with multi UAVs, the state space is consists of the the state of ego UAV s_{ego}, the state of surrounding UAVs s_{dyn}, the state of destination s_{des}, and the state of surrounding static obstacles such as walls or goods in the warehouse s_{sta}. The state of s_{ego} and s_{dyn} include current position coordinates, speed and direction, so they can be represented by vector $[x, y, v_x, v_y, cos\theta, sin\theta]$, and the state of static entity of s_{des} and s_{sta} only include position coordinates $[x, y]$. Note that the number of dynamic obstacles n_1 and static obstacles n_2 depended on the field of

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

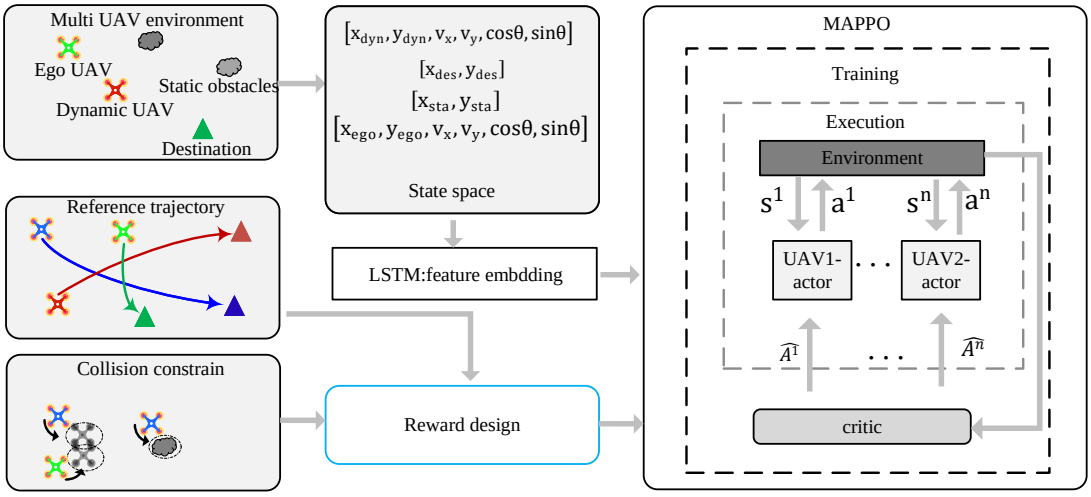


Fig. 5: The MAPPO based MAPF training framework

each UAV's view size, so the total state space of every UAV is $\{s_{ego}, s_{des}, s_{obs}\}$, where $s_{obs} = \{s^i_{dyn}, s^j_{sta} | i \in n_1, j \in n_2\}$.

Action space: As mentioned in Section III, the moving space of UAV is supposed to be a 2D space and the UAV moves in a continuous motion, according to the motion model of UAV, the input to UAV control module is yaw angle and acceleration. So the action space of UAV can be defined as

$$A = \{(\phi, acc) | \phi \in [-1, 1], acc \in -1, 1\}, \tag{12}$$

where the range $[-1, 1]$ are dimensionless representation, they can be mapped into actual ranges such as $[-\frac{\pi}{2}, \frac{\pi}{2}]$ and $[-10m/s^2, 10m/s^2]$, which depend on the UAV's actual hardware configurations such as quality and power.

Reward function: The purpose of reward function is to encourage all the UAVs to get into target task locations or destinations as quickly as possible and keep in safe and rational states, it will determine the action mode with different observations, which is defined as

$$R = R_c + R_d, \tag{13}$$

where R_c is collision reward, and R_d is distance reward. Collision reward includes collision with static obstacles such as goods, walls and dynamic obstacles such as other flying UAVs, so it is defined as

$$R_c = \lambda_1 R_{c_s} + \lambda_2 R_{c_d}. \tag{14}$$

The distance rewards R_d are consist of two parts, one is current distance from corresponding destinations, another is reference trajectory distance from current positions

$$R_d = \lambda_3 R_{des} + \lambda_4 R_{ref}. \tag{15}$$

Inspired by previous work [17], the reference trajectory are designed to improve the convergence process of training, for in the environment without restraint, the explore space in training environment is very large, the reference trajectory can be used to speed up training. The calculation of the reference trajectory is included in the training process. In order not to affect the training speed, it is necessary to use a path finding algorithm with a faster search speed, such as JSP. The reference trajectory τ_{ref} is composed of a set of trajectory points, and when calculating this part of the reward, the closest distance from the current position P to the entire trajectory point P_τ can be calculated as the reward

$$R_{d_ref} = min\{d_i | d_i = dis(P, P_\tau), P_\tau \in \tau_{ref}\}. \tag{16}$$

The reward design needs to drive the UAV toward the destination at the fastest speed while avoiding collisions with other UAVs and static obstacles, the design philosophyas is shown in Fig. 6. The most important thing in the reward design is to take the safety into account, that is the UAV cannot collide with other UAVs and avoid obstacles on the wall or any other static obstacles in the warehouse. On the basis of safety, the UAV needs to move towards the destination location as soon as possible. The direction of the destination may not be the optimal path direction, so the UAV needs to move along the direction of the reference trajectory as much as possible, so the final reward function can be designed as follows:

$$R = \lambda_1 R_{c_d} + \lambda_2 R_{c_s} + \lambda_3 R_{d_des} + \lambda_4 R_{d_ref}, \tag{17}$$

where $\{\lambda_1 > \lambda_2 >> \lambda_3 > \lambda_4\}$.

2) Proximal Policy Optimization: Proximal policy optimization (PPO) is a policy gradient algorithm that aims to maximize expected reward by optimizing the parameters of the actor network. It uses a simple but effective objective function for mini-batch updates and overcome the deficiency of TRPO using a less complex first-order method. PPO is a family of first-order methods that incorporate additional techniques to ensure that new policies remain close to the old ones. PPO methods are simpler to implement and have been shown to perform better than TRPO in empirical studies.

The actor network π_θ is used to approximate the policy, and the critic network v_ϕ is employed to approximate the value function. The parameters of the actor network and the critic network are denoted by θ and ϕ, respectively. The clipped

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

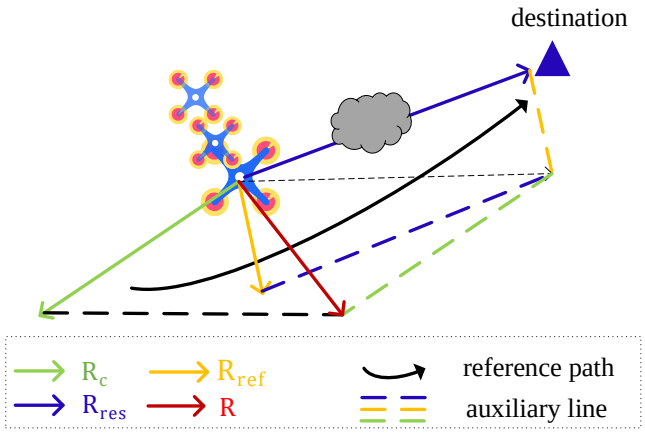


Fig. 6: View of reward design. UAV receives the drive from collision reward R_c, distance reward R_{des} and reference reward R_{ref}. R_c drives UAV to move in the opposite direction from the obstacle, R_{des} drives UAV to move in the direction of destination, R_{ref} drives the UAV to move in the direction of the reference trajectory, and the final total direction of motion is R.

surrogate objective function of PPO is formulated as

$$L^{\mathrm{CLIP}}(s,a,\theta) = E_t\Big[\min(r(\theta), \mathrm{clip}(\theta,\varepsilon))\widehat{A}(t)\Big], \tag{18}$$

where

$$\mathrm{clip}(\theta,\varepsilon) = \begin{cases} 1-\varepsilon, & r(\theta) < 1-\varepsilon, \\ r(\theta), & 1-\varepsilon < r(\theta) < 1+\varepsilon, \\ 1+\varepsilon, & 1+\varepsilon < r(\theta), \end{cases} \tag{19}$$

and

$$r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\mathrm{k}}}(a|s)}, \tag{20}$$

$\pi_{\theta_{\mathrm{k}}}(a|s)$ is old policy and ε is a small hyper-parameter which roughly says how far away the new policy is allowed to go from the old. To prevent modify of the objective value excessively, the clipping method is adopted. The generalized advantage estimator (GAE) $\widehat{A}(t)$ is defined as

$$\widehat{A}(t) \approx r(s_t,a_t) + \gamma v_\phi(s_{t+1},a_{t+1}) - v_\phi(s_t,a_t). \tag{21}$$

PPO updates actor network's parameters with clip method by

$$\theta_{k+1} = \arg\max_\theta \mathrm{E}\left[L^{\mathrm{CLIP}}(s,a,\theta)\right]. \tag{22}$$

The critic network is parameterized by ϕ and the loss function is defined as

$$L^V(s,a,\phi) = E_t\Big[\big(v_\phi(s_t,a_t) - y(t)\big)^2\Big], \tag{23}$$

where $y(t) = R_t + \gamma v_\phi(s_{t+1},a_{t+1})$ is temporal difference target value, R_t is the reward at current environment. PPO updates actor network's parameters by

$$\phi_{k+1} = \arg\min_\phi \mathrm{E}\left[L^V(s,a,\phi_k)\right]. \tag{24}$$

3) MAPPO Network: MAPPO is also composed of n actor and one critic networks as shown in Fig. 5, where the critic network is a globally shared network that only participates in the training process. The purpose of training is to obtain an actor network that can output actions. For all agents, the structure and parameters of the actor network are the same, but their received inputs are different, as the state spaces of different agents are different. In order to encode the temporal information of the surrounding state of each UAV, we use a long short term memory (LSTM) network to prepossess the surrounding environment over a period of time.

During the centralized training stage, the critic network serves as a central coordinator, computing the centralized value $v_\phi(s,a_1,a_2,\ldots,a_u)$ using the global state information that incorporates actions and observations from all agents. The centralized critic network v_ϕ assesses the actions of actors from a global viewpoint and directs the selection of optimal actions. Subsequently, the parameters ϕ is updated by the critic network to minimize the loss

$$\mathrm{L}^V_{MAPPO}(\phi) = E\Big[v_\phi(s_t,a_t^1,a_t^2,\ldots,a_t^n) - y(t)\Big]^2, \tag{25}$$

where

$$y(t) = R_t + \gamma v_\phi\left(s_{t+1},a_{t+1}^1,a_{t+1}^2,\ldots,a_{t+1}^n\right), \tag{26}$$

and $R_t = \sum_{i=0}^n (R_t^i)$ is total reward of all agents, $s_t = \{s_t^1, s_t^2, \ldots, s_t^n\}$ and ϕ is the parameter of the critic network. Besides, all agents' actor network are the same, so all agents will update the actor network in turn according to the centralized $\widehat{A}(t)$ based on the critic network and its observation, or they can update the actor network in batch method at the same time. In specific, the agent i's actor network will adjust the parameters θ directly with the clip objective function that is similar to (18) and (22):

$$\theta_{k+1} = \arg\max_\theta \mathrm{E}\left[L^{\mathrm{CLIP}}(s^i,a^i,\theta)\right]. \tag{27}$$

In the decentralized execution phase, without the need for a critic network, only the actor network runs online. Each complementary and relevant actor makes real-time decisions based solely on local observations. If the actor network is effectively trained, theoretically each agent can obtain the globally optimal action without need to access information from other agents. Furthermore, any number of UAVs can adopt the same action policy, and it is not necessary to match the number of UAVs in the training phase.

4) State Space Preprocessing: In order to better capture the environmental information around each UAV, each UAV obtains information about the current state around time t, while storing the state space information of the previous n moments. The LSTM network is used to encode the surrounding environmental information and the output of LSTM is fed to MAPPO algorithm. The calculation process of x_t is as follows:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f), \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \\ \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C), \\ C_t &= f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t, \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \\ h_t &= o_t \cdot \tanh(C_t), \end{aligned} \tag{28}$$

where C is the unit state, h is the hidden layer state, o_t is the output, σ is the sigmoid function, $tanh$ is the hyperbolic

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

Algorithm 1 Training Algorithm of MAPPO

1: Initialize policy network π_θ, value network v_ϕ, data buffer D, let $\pi_{\theta'} = \pi_\theta$
2: **for** episode = $1, ..., N_{eps}$ **do**
3: //Collect data
4: **for** agent = $1, ..., n$ **do**
5: Collect data (s_t, a_t, r_t) for T steps, which denote
6: state, action and reward of step t, respectively.
7: Insert data (s, a, r) into buffer D
8: **end for**
9: // Network update
10: **for** k = $1, ..., K_{iter}$ **do**
11: Shuffle data's order in buffer D
12: Compute advantages $\widehat{A}(t)$ based on (21)
13: Compute temporal difference $y(t)$ based on (26)
14: **Update actor:**
15: Apply gradient ascent on θ based on (27)
16: **Update critic:**
17: Apply gradient descent on ϕ based on (24)
18: **end for**
19: **end for**

tangent function, W and b are learning parameters. The LSTM $f_{lstm}(x_i, x_{i+1}, x_{i+2}...)$will take n state as inputs and every input corresponds to one unit as in (28), so the MAPPO state space at time t after LSTM preprocessing is:

$$S^t = \{s^t_{ego}, s^t_{des}\} + f_{lstm}(s^{t-p}_{obs}, s^{t-p+1}_{obs}, ..., s^t_{obs}), \quad (29)$$

where p is the previous time step of surrounding obstacles used.

5) Training Algorithm: The PPO-based multi-agent reinforcement learning algorithm for multi-UAV path finding is shown in Algorithm 1. Specifically, the training process of reinforcement learning includes experience collection stage and network updates stage. In the experience collection stage, before maximum time step T is reached, each UAV makes decisions using the same policy network $\pi(\theta)$ and do experience collection (s, a, r) (line 5-6), and then inserts the collected data into buffer D for network update (line 7). In the second stage, K_{iter} iteration every episode will be done to update network parameters. First, the algorithm will randomly renumber and shuffle the data samples to break any correlation between them and ensure the stability of the training process (line 11), and then the advantage estimation and temporal difference value are calculated based on (21) and (26) (lines 12-13). And then gradient ascent on θ and gradient descent on ϕ are applied based on (27) and (24), respectively (lines 14-17).

VI. SIMULATION RESULTS

In simulation experiments, we test the performance of our proposed joint path finding framework assisted with reinforcement learning, including solving time, the total path length of task allocation and path finding, and planning success rate. Secondly, we also compare the algorithm based on MAPPO used in this paper with other MARL algorithms. Finally, we conduct ablation experiments on the LSTM state space prepossessing proposed in this paper to verify its effectiveness.

TABLE I: Training Parameters Setup

Parameters	Values
number of parallel envs	100
mini batch size	8
steps	100M
episodes length	40
MLP hidden size	[128,128]
active function	relu
optimizer	Adam
ppo epsilon clip	0.2
γ in GAE	0.99
$\lambda_1, \lambda_2, \lambda_3, \lambda_4$	10,5,1,0.8
time steps of LSTM	2

A. Simulation Setting

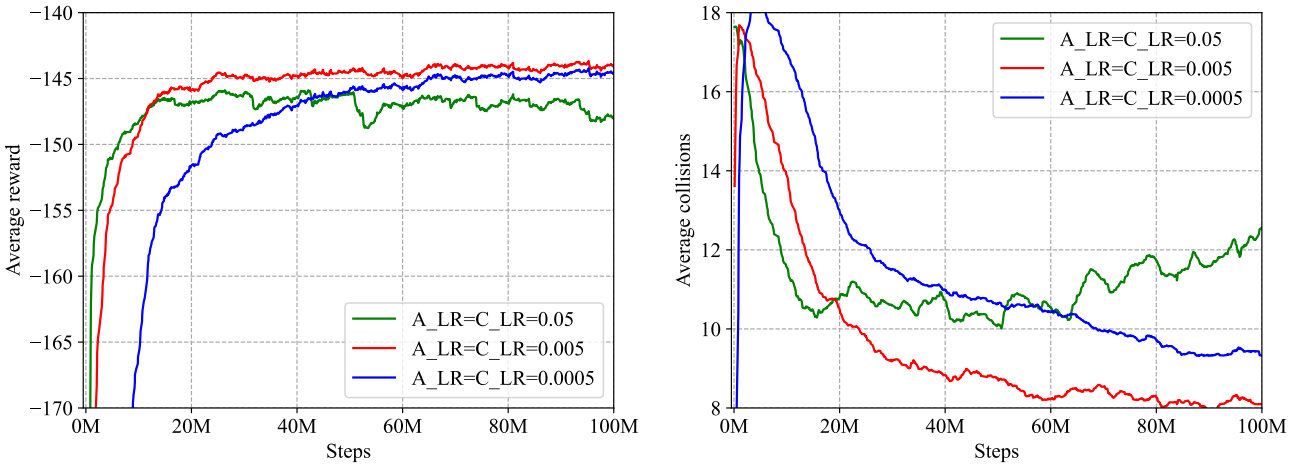
All simulation test codes are implemented using Python, and the overall framework and multi-UAV environment are based on openAI Gym Env framework. The environment is a 20m×20m map to simulate the layout of a warehouse, with a diameter of 2m for each target task location. The obstacle avoidance range of the UAV is 1m. The TSP problem of the task allocation module is solved by the LKH-2.0.9 [40] TSP solver, while the neural network module is implemented using pytorch. All training and testing environments are implemented in the Ubuntu 20.04 system, with a 13th Gen Intel (R) Core (TM) i9-13900KF CPU, 3.00 GHz, 64.0GB RAM, and NVIDIA RTX 4080 GPU. The parameters used in the training process of the multi-agent path finding module are shown in Table I.

B. Convergence Performance

In the path finding module, we only need to find a conflict-free path from the starting location to the destination location, without considering task allocation. Therefore, we train the MAPPO based path finding algorithm in a random map environment (where the starting and destination positions of the UAVs are random) with 5 UAVs to obtain the optimal conflict-free path finding policy. During the training process, all UAVs randomly explore the entire environment, and when the maximum number of exploration steps is reached, the scene will be reset. The learning rates of actor and critic networks can affect the convergence of neural networks, which is represented by the average reward of each epoch and the average number of collisions between UAVs and obstacles. We tested different learning rates, and the result of average reward and average path conflict times is shown in Fig. 7a and Fig. 7b. In the initial phase of training, the UAVs hardly move, there are almost no collisions. But as the training progresses, all UAVs start to move, so the number of UAV collisions increases rapidly in the initial phase. A good training result should correspond to a larger reward and a smaller number of conflicts. From the result, it can be seen that A_LR=C_LR=0.05 has a fast convergence

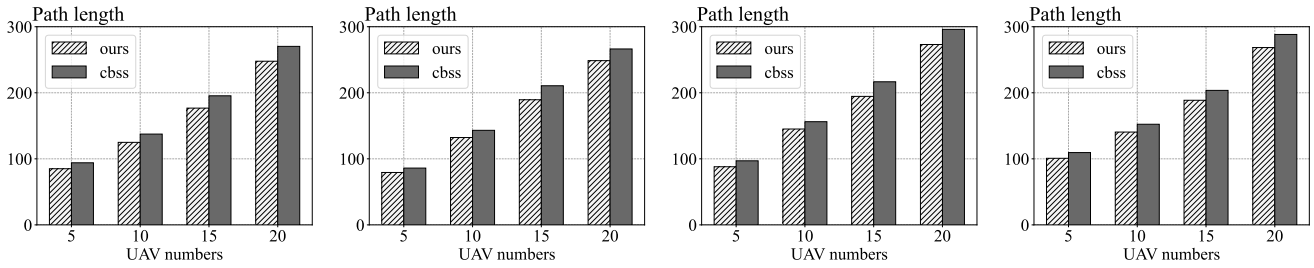
Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152



(a) (b)

Fig. 7: Convergence of MAPPO



(a) M=5 (b) M=10 (c) M=15 (d) M=20

Fig. 8: Path length compared with CBSS

speed, but the average reward value of the final convergence reward is small and the average number of conflicts is large. When A_LR=C_LR=0.0005, the learning rate is set too small, resulting in non-convergence of model performance. When A_LR=C_LR=0.005, the final convergence yields the maximum reward value and the minimum average number of conflicts. Therefore, in subsequent performance and effect comparison experiments, the path finding policy trained at this learning rate is adopted.

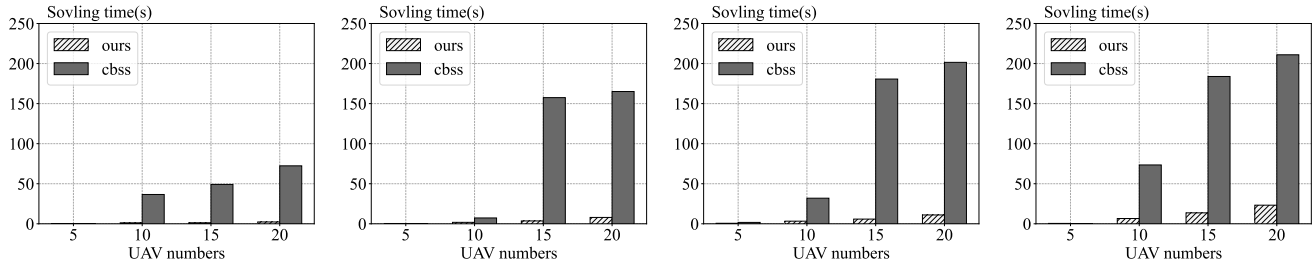
C. Comparison with Heuristic Algorithms

In order to verify the scalability of the path finding module proposed in this paper, i.e., the trained actor network can be used in any scenario with a large number of UAVs without retraining, the previous path finding actor network trained in the scenario of 5 UAVs is used in the environment with more UAVs. We compare the framework composed of the task planning module and the trained path finding module in this paper with the latest combinatorial path finding algorithm CBSS [10] in terms of solution time and solution quality. The task planning module in CBSS adopts the same algorithm based on mTSP as that in this paper, but the optimal task planning result may not be used in CBSS. In the path finding stage of CBSS, if the path finding fails, the task planning will be redone, and the suboptimal task planning result will be used to meet the time constraint of the solver. The parameters of CBSS are set to $\epsilon = 0$, time_limit=20 minutes, indicating that it will find an optimal path but within a 20-minute constraint solving time, so there is a probability of path finding failure. We tested the time solving efficiency and the total path length obtained by solving the number of UAVs in configurations of M=5, M=10, M=15, and M=20, as well as the number of tasks in configurations of 5, 10, 15, and 20 in a random environment (where the starting locations, destination locations, task target locations and the mode of target constrained function f_A are randomly generated).

The solution quality is represented by the sum of the conflict paths of all UAVs output by the algorithm, where the shorter the path, the better the solution quality. The comparison of solution quality between our framework and CBSS at different UAV numbers and task target numbers is shown in Fig. 8, from the result it can be seen that our

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152



(a) M=5 (b) M=10 (c) M=15 (d) M=20

Fig. 9: Time used compared with CBSS

framework can obtain a shorter path results in any scenario, because compared to the CBSS of heuristic algorithms, the framework proposed in this paper will strive to ensure the optimality of task planning and path finding results as much as possible. CBSS is a heuristic algorithm that may not yield the optimal solution. In Fig. 9, the comparison between our proposed framework and the CBSS algorithm in terms of solving time is shown. It can be seen that the proposed framework has a faster solving speed and does not grow as quickly as the task size and the number of UAV increases. This is because in path finding, although the actor network has undergone a long time of training, this training process is one-time, each subsequent UAV only needs to use the actor network for inference, the time complexity of the inference process is constant and the running efficiency is very high. The comparison of success rates between our framework and the CBSS algorithm is shown in Fig. 10. The path finding module of our framework is based on learning method, as shown in Fig. 7b, although the training is sufficient, the algorithm proposed in this paper is a reinforcement learning algorithm based on neural network, the neural network has uncertainty, and the conflict-free design of the reward function is a soft constraint, which cannot completely guarantee the safety of the actions taken by the UAV, so there is still possibility of UAVs path conflicts and solution failure of the proposed framework. In some scenarios, the CBSS algorithm may also fail to get a conflict-free solution. From Fig. 10, it can be seen that even though the path finding policy of our framework is distributed for each UAV, the success rate is relatively high compared to CBSS.

D. Comparison with Other MARL Algorithms

MAPPO is used in our framework as the training algorithm for the MAPF module. We compare it with the existing popular MADDPG algorithm to demonstrate the advantages of the proposed algorithm in path finding. All training parameters of MADDPG and MAPPO are the same. Fig. 11a and Fig. 11b show the convergence results of MAPPO and MADDPG in reward and path conflict times during the training process. It can be seen that the path finding module based on MAPPO in this paper has better convergence speed and can achieve better reward and collision times than MAPPDG. Although MADDPG also adopts centralized training with a decentralized execution framework, its ability to explore deterministic policies is poor and there is a high probability to fall into the local optimal solution. Besides, MADDPG is prone to overestimate the value of the Q function, which can lead to unstable training processes. MAPPO also adopts an actor network with local and global observations to achieve collaboration between UAVs, and adds action entropy rewards to encourage UAV exploration. Therefore, MAPPO can receive a higher reward compared to MADDPG.

E. Ablation Experiment

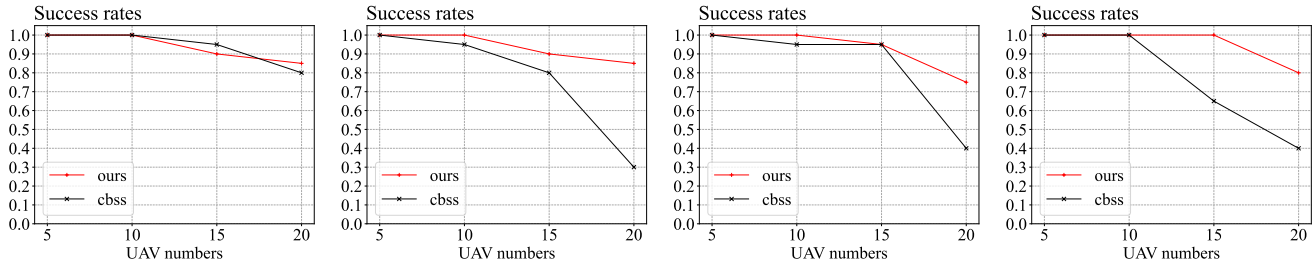
This paper uses an LSTM network to capture temporal information of the surrounding environment of each UAV to preprocess the surrounding environment as input to the actor network. Fig. 11c and Fig. 11d show the convergence of the MAPPO with LSTM and MAPPO without LSTM in terms of reward and collision times, respectively. The LSTM network encodes the information of the UAV's surrounding environment and can help UAV to make path planning according to the motion state of other obstacles around it in the past period of time. Therefore, the introduction of the LSTM network takes into account more information, which can accelerate the convergence speed of the neural network and ultimately improve the performance of the model.

VII. Conclusions

In this paper, a reinforcement learning assisted joint path finding framework is proposed to solve the task allocation and MAPF problems in a warehouse cargo detection environment based on multiple heterogeneous UAVs. The framework consists of a centralized task allocation module and a distributed UAV path finding policy, which meets the design requirements of a multi-functional IIoT system. The centralized task allocation module based on mTSP transformation algorithm is responsible for assigning tasks according to the functions and states of all UAVs with the goal of minimizing the total path. The path finding policy is based on a partially observable Markov model to construct a multi-agent reinforcement learning policy, which can be distributed and deployed to

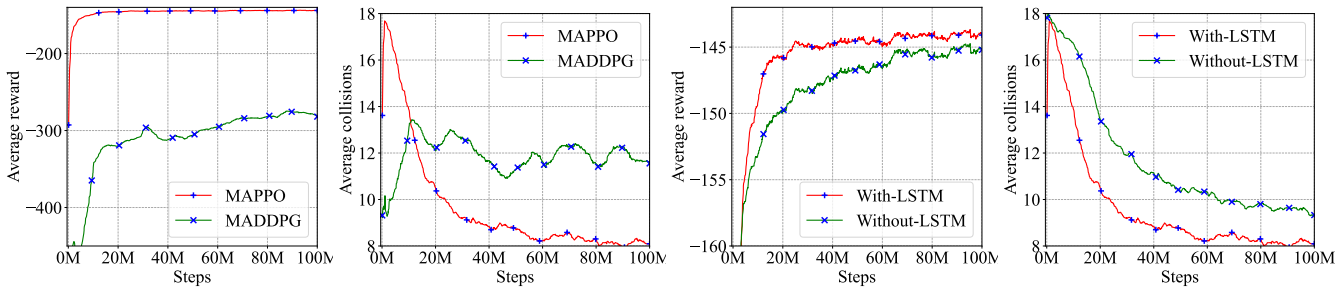
Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152



(a) M=5 (b) M=10 (c) M=15 (d) M=20

Fig. 10: Conflict rates compared with CBSS



(a) (b) (c) (d)

Fig. 11: Convergence with different MARL algorithms and with or without-LSTM

any number of UAVs. The simulation results show that the proposed framework has advantages over existing state of art algorithms in terms of solving efficiency, solving quality, success rate and scalability.

REFERENCES

[1] H. Menouar, I. Guvenc, K. Akkaya, A. S. Uluagac, A. Kadri, and A. Tuncer, "UAV-enabled intelligent transportation systems for the smart city: Applications and challenges," *IEEE Communications Magazine*, vol. 55, no. 3, pp. 22–28, Mar. 2017.

[2] K. Harikumar, J. Senthilnath, and S. Sundaram, "Multi-UAV oxyrrhis marina-inspired search and dynamic formation control for forest firefighting," *IEEE Transactions on Automation Science and Engineering*, vol. 16, no. 2, pp. 863–873, Sep. 2019.

[3] H. Feng, J. Wang, Z. Fang, J. Chen, and D.-T. Do, "Evaluating aoi-centric harq protocols for UAV networks," *IEEE Transactions on Communications, (DOI: 10.1109/TCOMM.2023.3320696)*, 2023.

[4] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, "Industrial internet of things: Challenges, opportunities, and directions," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 11, pp. 4724–4734, Jul. 2018.

[5] R. Rumba and A. Nikitenko, "The wild west of drones: a review on autonomous-UAV traffic-management," in *International Conference on Unmanned Aircraft Systems (ICUAS)*, Athens, Greece, Jun, 2020, pp. 1317–1322.

[6] J. Wang, C. Jiang, Z. Han, Y. Ren, R. G. Maunder, and L. Hanzo, "Taking drones to the next level: Cooperative distributed unmanned-aerial-vehicular networks for small and mini drones," *IEEE Vehicular Technology Magazine*, vol. 12, no. 3, pp. 73–82, Jul. 2017.

[7] H. Kandath, T. Bera, R. Bardhan, and S. Sundaram, "Autonomous navigation and sensorless obstacle avoidance for UGV with environment information from UAV," in *IEEE International Conference on Robotic Computing (IRC)*, Orange County, CA, Jan, 2018, pp. 266–269.

[8] J. Wang, Z. Jiao, J. Chen, X. Hou, T. Yang, and D. Lan, "Blockchain-aided secure access control for UAV computing networks," *IEEE Transactions on Network Science and Engineering, (DOI: 10.1109/TNSE.2023.3324639)*, 2023.

[9] H. Feng, J. Wang, Z. Fang, J. Qian, and K.-C. Chen, "Age of information in UAV aided wireless sensor networks relying on blockchain," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 9, pp. 12 430–12 435, Apr. 2023.

[10] Z. Ren, S. Rathinam, and H. Choset, "Cbss: A new approach for multiagent combinatorial path finding," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2669–2683, Aug. 2023.

[11] J. Keller, D. Thakur, M. Likhachev, J. Gallier, and V. Kumar, "Coordinated path planning for fixed-wing UAS conducting persistent surveillance missions," *IEEE Transactions on Automation Science and Engineering*, vol. 14, no. 1, pp. 17–24, Dec. 2017.

[12] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI Magazine*, vol. 29, no. 1, pp. 9–9, Jul. 2008.

[13] Z. Ren, S. Rathinam, and H. Choset, "MS*: A new exact algorithm for multi-agent simultaneous multi-goal sequencing and path finding," in *IEEE International Conference on Robotics and Automation (ICRA)*, Xian, China, May, 2021, pp. 11 560–11 565.

[14] D. M. Saxena, S. Bae, A. Nakhaei, K. Fujimura, and M. Likhachev, "Driving in dense traffic with model-free reinforcement learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, Paris, France, May, 2020, pp. 5385–5392.

[15] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013.

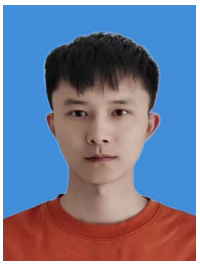
[16] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, Aug. 2023.

[17] Z. Liu, B. Chen, H. Zhou, G. Koushik, M. Hebert, and D. Zhao, "Mapper: Multi-agent path planning with evolutionary reinforcement learning in mixed dynamic environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, NV, Oct, 2020, pp. 11 748–11 754.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2024.3370152

[18] S. Chen, M. Wang, Y. Yang, and W. Song, "Conflict-constrained multi-agent reinforcement learning method for parking trajectory planning," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, London, UK, May, 2023, pp. 9421–9427.
[19] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. S. Kumar, S. Koenig, and H. Choset, "Primal: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2378–2385, Mar. 2019.
[20] H. Q. Ahmed and A. Yasar, "Intelligent bidirectional rapidly-exploring random trees for optimal motion planning in complex cluttered environments," *Robotics and Autonomous Systems*, vol. 68, pp. 1–11, Jun. 2015.
[21] Z. Yu, Z. Si, X. Li, D. Wang, and H. Song, "A novel hybrid particle swarm optimization algorithm for path planning of UAVs," *IEEE Internet of Things Journal*, vol. 9, no. 22, pp. 22 547–22 558, Jun. 2022.
[22] Z. Chen, J. Zhou, R. Sun, and L. Kang, "A new evolving mechanism of genetic algorithm for multi-constraint intelligent camera path planning," *Soft Computing*, vol. 25, pp. 5073–5092, Apr. 2021.
[23] W. Yue, W. Tang, L. Wang, Y. Xi, Z. Liu, and C. Zou, "Cooperative search algorithm for UAVs based on multi-ant colony," in *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Melbourne, Australia, Oct, 2021, pp. 1299–1304.
[24] J. Wang, C. Jiang, Z. Wei, C. Pan, H. Zhang, and Y. Ren, "Joint UAV hovering altitude and power control for space-air-ground IoT networks," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1741–1753, Oct. 2019.
[25] Y. Chen, D. Yang, and J. Yu, "Multi-UAV task assignment with parameter and time-sensitive uncertainties using modified two-part wolf pack search algorithm," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 6, pp. 2853–2872, Apr. 2018.
[26] C. Liu, Y. Guo, N. Li, and X. Song, "AoI-minimal task assignment and trajectory optimization in multi-UAV-assisted IoT networks," *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21 777–21 791, Jun. 2022.
[27] X. Wang, T.-M. Choi, H. Liu, and X. Yue, "Novel ant colony optimization methods for simplifying solution construction in vehicle routing problems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 11, pp. 3132–3141, Apr. 2016.
[28] J. Zhuqing, M. Kai, R. Yiling, W. Peng, Z. Hongkai, and W. Shuihua, "A path planning method using adaptive polymorphic ant colony algorithm for smart wheelchairs," *Journal of Computational Science*, vol. 25, pp. 50–57, Mar. 2018.
[29] X. Chengke, C. Danfeng, L. Di, Z. Zheng, and L. Lian, "Path planning of multiple autonomous marine vehicles for adaptive sampling using voronoi-based ant colony optimization," *Robotics and Autonomous Systems*, vol. 115, pp. 90–103, May 2019.
[30] H. Ma, J. Li, T. Kumar, and S. Koenig, "Lifelong multi-agent path finding for online pickup and delivery tasks," 2017. [Online]. Available: arXiv:1805.11074.
[31] M. Liu, H. Ma, J. Li, and S. Koenig, "Task and path planning for multi-agent pickup and delivery," in *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, Montreal, Canada, May, 2019, p. 1152–1160.
[32] Q. Xu, J. Li, S. Koenig, and H. Ma, "Multi-goal multi-agent pickup and delivery," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Kyoto, Japan, Oct, 2022, pp. 9964–9971.
[33] C. Henkel, J. Abbenseth, and M. Toussaint, "An optimal algorithm to solve the combined task allocation and path finding problem," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Macau, China, Nov, 2019, pp. 4140–4146.
[34] J. Wang, L. Bai, J. Chen, and J. Wang, "Starling flocks-inspired resource allocation for ISAC-aided green ad hoc networks," *IEEE Transactions on Green Communications and Networking*, vol. 7, no. 1, pp. 444–454, Jan. 2023.
[35] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, and C. Cadena, "From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, Singapore, May, 2017, pp. 1527–1533.
[36] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, Canada, Sep, 2017, pp. 31–36.
[37] S. Ouahouah, M. Bagaa, J. Prados-Garzon, and T. Taleb, "Deep-reinforcement-learning-based collision avoidance in UAV environment," *IEEE Internet of Things Journal*, vol. 9, no. 6, pp. 4015–4030, Oct. 2022.
[38] P. Oberlin, S. Rathinam, and S. Darbha, "Today's traveling salesman problem," *IEEE Robotics Automation Magazine*, vol. 17, no. 4, pp. 70–77, Dec. 2010.
[39] P. Oberlin, S. Rathinam, and S. Darbha, "A transformation for a heterogeneous, multiple depot, multiple traveling salesman problem," in *American Control Conference (ACC)*, St. Louis, Missouri, Jun, 2009, pp. 1292–1297.
[40] K. Helsgaun, "General k-opt submoves for the Lin–Kernighan TSP heuristic," *Mathematical Programming Computation*, vol. 1, pp. 119–163, Jan. 2009.



Guodong Zhao (Student Member, IEEE) received his B.S. degree in Information and computational science from the Shandong University of Science and Technology, Qingdao, China in 2018, and the M.S. degree in computer system architecture from Northeastern University, Shenyang, China in 2021. From 2021 to 2023, he worked as a senior R&D engineer for AI high performance computing at Baidu. He is currently pursuing the Ph.D. degree in aircraft design from Beihang University, Beijing,China. His research interests lie in the area of UAV and AI.



Ye Wang received his B.S. degree in computer science and technology from Henan Normal University, Xinxiang, China in 2022. He is currently pursuing the M.S. degree in the School of Computer and Artificial Intelligence from Zhengzhou University, Zhengzhou, China. His research interests lie in deep reinforcement learning and swarm intelligence.



Tong Mu received the B.Eng. degree in electrical engineering and automation from the Qingdao City University, Qingdao, China, in 2017, and the M.Eng. degree in control science and engineering from the Qingdao University of Technology, Qingdao, China, in 2020. He is currently pursuing the Ph.D. degree in control science and engineering with the North China University of Technology, Beijing, China. He current research interests include networked control systems, networked predictive control, and data-driven control.



Zhijun Meng received the Ph.D. degree from the School of Aeronautic Science and Engineering, Beihang University, China in 2009. He is currently a professor in aerospace science and engineering with Beihang University. His research interests lie in the area of autonomous flight and collaborative exploration of UAV.



Zichen Wang received his B.S. degree in Vehicle Engineering from the Beijing Institute of Technology, Beijing, China in 2020. He is currently pursuing the Ph.D. degree in the School of Aeronautic Science and Engineering from Beihang University, Beijing, China. His research interests lie in the area of UAV path planning and autonomous exploration.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on May 29,2024 at 08:40:37 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.